Snowman Merkle Airdrop Audit Report

Version 1.0

SymmaTe

March 8, 2026

Snowman Merkle Airdrop Audit Report

SymmaTe

March 8, 2026

Prepared by: SymmaTe (https://github.com/SymmaTe) Lead Security Researcher: - SymmaTe

Table of Contents

  - [L-1] `SnowmanAirdrop::claimSnowman` never checks `s_hasClaimedSnowman`, allowing multiple claims
  - [L-2] `Snow::s_earnTimer` is a global variable, allowing any user to reset the earn cooldown for everyone

- Informational
  - [I-1] `Snow::s_buyFee` should be declared `immutable`
  - [I-2] Missing getter functions for key state variables in Snow
  - [I-3] `SnowmanAirdrop::s_claimers` array is declared but never used
  - [I-4] `Snow::collectFee` uses bare `transfer` instead of `safeTransfer`
  - [I-5] `Snowman::SM__NotAllowed` error is declared but never used

Protocol Summary

Snowman Merkle Airdrop is a protocol that distributes Snowman NFTs to whitelisted users via a Merkle Tree-based airdrop. Users first accumulate Snow ERC20 tokens through farming or purchasing, then claim Snowman ERC721 NFTs by submitting a Merkle proof and an EIP-712 signature. The airdrop can be claimed on behalf of a user by a third party (gasless claim pattern).

Disclaimer

The SymmaTe team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
6efcb0f
```

Scope

```
src/
--- Snow.sol
--- Snowman.sol
--- SnowmanAirdrop.sol
```

Roles

- Owner: Deploys the contracts and sets the Merkle root.
- Whitelisted users: Addresses included in the Merkle tree who can claim Snowman NFTs.
- Third-party callers: Anyone can submit a claim on behalf of a whitelisted user (gasless pattern).

Executive Summary

Category	Details
Solidity Version	0.8.24
Chain(s)	Ethereum
nSLOC	215

Issues found

Severity	Number of issues found
Critical	1
High	1
Medium	0
Low	2
Informational	5
Total	**9**

Findings

Critical

[C-1] `Snowman::mintSnowman` has no access control, allowing anyone to mint unlimited NFTs

Description: The `Snowman::mintSnowman` function is marked `external` with no access restriction. The entire purpose of `SnowmanAirdrop` is to gate NFT minting behind Merkle proof and EIP-712 signature verification, yet anyone can bypass this by calling `mintSnowman` directly on the Snowman contract.

```
    function mintSnowman(address receiver, uint256 amount) external {
@>      // @audit - No access control whatsoever
        for (uint256 i = 0; i < amount; i++) {
            _safeMint(receiver, s_TokenCounter);
            emit SnowmanMinted(receiver, s_TokenCounter);
            s_TokenCounter++;
        }
    }
```

Impact: Any user can mint an unlimited number of Snowman NFTs for free, completely bypassing the airdrop mechanism. The NFT has no scarcity and the protocol's core invariant is broken.

Proof of Concept: Add the following test to `TestSnowmanAirdrop.t.sol`:

Code

```
function test_anyoneCanMintDirectly() public {
    address attacker = makeAddr("attacker");
```

```
    // Attacker calls mintSnowman directly, no Snow tokens needed
    vm.prank(attacker);
    nft.mintSnowman(attacker, 1000);

    // Attacker now holds 1000 NFTs without going through the airdrop
    assertEq(nft.balanceOf(attacker), 1000);
    assertEq(nft.getTokenCounter(), 1000);
}
```

Recommended Mitigation: Restrict `mintSnowman` so only the `SnowmanAirdrop` contract can call it. Store the airdrop address as an immutable in the constructor and add an explicit check.

```
+   address private immutable i_airdrop;
+   error SM__NotAllowed();

    constructor(address airdrop) {
+       i_airdrop = airdrop;
    }

    function mintSnowman(address receiver, uint256 amount) external {
+       if (msg.sender != i_airdrop) revert SM__NotAllowed();
        for (uint256 i = 0; i < amount; i++) {
            _safeMint(receiver, s_TokenCounter);
            emit SnowmanMinted(receiver, s_TokenCounter);
            s_TokenCounter++;
        }
    }
```

High

[H-1] MESSAGE_TYPEHASH has a typo (“addres” instead of “address”), breaking EIP-712 signature verification for all users

Description: The `MESSAGE_TYPEHASH` constant is computed with a misspelled type string — `"addres"` instead of `"address"`. This causes the on-chain typehash to differ from what any standard EIP-712 compliant wallet produces.

```
@>  bytes32 private constant MESSAGE_TYPEHASH =
↪   keccak256("SnowmanClaim(addres receiver, uint256 amount)");
//
↪   ^^^^^^ missing 's'
```

Impact: All signatures generated by standard wallets using the correct type string `"SnowmanClaim(address receiver, uint256 amount)"` will fail the `_isValidSignature` check. No legitimate user can claim their Snowman NFT.

Proof of Concept: Add the following test to `TestSnowmanAirdrop.t.sol`:

Code

```
function test_correctSignatureFails() public {
    // Compute the CORRECT typehash (proper EIP-712 spelling)
    bytes32 correctTypehash = keccak256("SnowmanClaim(address receiver,
↪   uint256 amount)");
    bytes32 wrongTypehash   = keccak256("SnowmanClaim(addres receiver,
↪   uint256 amount)");

    // They are different — the contract uses the wrong one
    assertNotEq(correctTypehash, wrongTypehash);

    // A wallet signing with the correct typehash produces a digest
    // that the contract cannot verify, causing claimSnowman to revert
    uint256 amount = snow.balanceOf(alice);
    bytes32 structHash = keccak256(abi.encode(correctTypehash, alice,
↪   amount));
    bytes32 correctDigest = keccak256(
        abi.encodePacked("\x19\x01", airdrop.DOMAIN_SEPARATOR(),
↪   structHash)
    );

    (uint8 v, bytes32 r, bytes32 s) = vm.sign(alKey, correctDigest);

    vm.prank(alice);
    snow.approve(address(airdrop), amount);

    // Reverts with SA__InvalidSignature — correct signature rejected
    vm.expectRevert(SnowmanAirdrop.SA__InvalidSignature.selector);
    vm.prank(satoshi);
    airdrop.claimSnowman(alice, AL_PROOF, v, r, s);
}
```

Recommended Mitigation: Fix the typo in the type string.

```
-   bytes32 private constant MESSAGE_TYPEHASH =
↪   keccak256("SnowmanClaim(addres receiver, uint256 amount)");
+   bytes32 private constant MESSAGE_TYPEHASH =
↪   keccak256("SnowmanClaim(address receiver, uint256 amount)");
```

Medium

None.

Low

[L-1] `SnowmanAirdrop::claimSnowman` never checks `s_hasClaimedSnowman`, allowing a user to claim multiple times

Description: The mapping `s_hasClaimedSnowman` is set to `true` after a successful claim but is never read before processing a new claim. After claiming, a user's Snow balance drops to zero. However, they can re-acquire the same amount of Snow and re-submit the identical Merkle proof to claim again.

```
    function claimSnowman(...) external nonReentrant {
@>      // @audit - s_hasClaimedSnowman[receiver] is never checked here
        ...
        i_snow.safeTransferFrom(receiver, address(this), amount);
        s_hasClaimedSnowman[receiver] = true;   // set but never read on
↪   entry
        i_snowman.mintSnowman(receiver, amount);
    }
```

Impact: A user can repeatedly claim: buy/earn Snow → claim Snowman NFTs → buy/earn Snow again → claim again. The Snowman NFT supply can be drained by any eligible user.

Proof of Concept: Add the following test to `TestSnowmanAirdrop.t.sol`:

Code

```
function test_multipleClaimsAllowed() public {
    // --- First claim ---
```

```
        vm.prank(alice);
        snow.approve(address(airdrop), 1);
        bytes32 digest1 = airdrop.getMessageHash(alice);
        (uint8 v1, bytes32 r1, bytes32 s1) = vm.sign(alKey, digest1);

        vm.prank(satoshi);
        airdrop.claimSnowman(alice, AL_PROOF, v1, r1, s1);

        assertEq(nft.balanceOf(alice), 1);
        assertTrue(airdrop.getClaimStatus(alice)); // marked as claimed

        // --- Alice re-acquires Snow ---
        vm.warp(block.timestamp + 1 weeks);
        vm.prank(alice);
        snow.earnSnow(); // balance restored to 1

        // --- Second claim with same proof ---
        vm.prank(alice);
        snow.approve(address(airdrop), 1);
        bytes32 digest2 = airdrop.getMessageHash(alice);
        (uint8 v2, bytes32 r2, bytes32 s2) = vm.sign(alKey, digest2);

        vm.prank(satoshi);
        airdrop.claimSnowman(alice, AL_PROOF, v2, r2, s2); // succeeds again!

        // Alice claimed twice despite s_hasClaimedSnowman being true
        assertEq(nft.balanceOf(alice), 2);
    }
```

Recommended Mitigation: Add the claimed check at the start of `claimSnowman`.

```
+   error SA__AlreadyClaimed();

    function claimSnowman(...) external nonReentrant {
+       if (s_hasClaimedSnowman[receiver]) revert SA__AlreadyClaimed();
        if (receiver == address(0)) revert SA__ZeroAddress();
        ...
    }
```

[L-2] `Snow::s_earnTimer` is a global variable, allowing any user to reset the earn cooldown for everyone

Description: `s_earnTimer` is a single `uint256` shared across all users. Both `buySnow()` and `earnSnow()` write to it, meaning any user's interaction resets the cooldown for every other user.

```
    function buySnow(uint256 amount) external payable canFarmSnow {
        ...
@>      s_earnTimer = block.timestamp; // resets cooldown for ALL users
    }

    function earnSnow() external canFarmSnow {
        if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks))
        ↪  {
            revert S__Timer();
        }
        _mint(msg.sender, 1);
@>      s_earnTimer = block.timestamp; // resets cooldown for ALL users
    }
```

Impact: An attacker can call `buySnow(1)` every 6 days (spending only the minimum fee) to permanently prevent all users from ever successfully calling `earnSnow()`. The free-claim mechanism becomes permanently unusable.

Proof of Concept: Add the following test to `TestSnow.t.sol`:

Code

```
function test_buySnowBlocksAllEarnSnow() public {
    address attacker = makeAddr("attacker");
    address victim   = makeAddr("victim");

    // Give attacker WETH to buy Snow
    deal(address(weth), attacker, 1000 ether);
    vm.prank(attacker);
    weth.approve(address(snow), type(uint256).max);

    // Victim successfully earns Snow at T=0
    vm.prank(victim);
    snow.earnSnow();
    assertEq(snow.balanceOf(victim), 1);

    // Attacker buys 1 Snow — resets global s_earnTimer to now
    vm.warp(block.timestamp + 1 weeks);
```

```
        vm.prank(attacker);
        snow.buySnow(1);

        // Victim tries to earn — reverts because attacker reset the timer
        vm.expectRevert(Snow.S__Timer.selector);
        vm.prank(victim);
        snow.earnSnow();

        // Attacker repeats every 6 days to keep the block permanent
        vm.warp(block.timestamp + 6 days);
        vm.prank(attacker);
        snow.buySnow(1); // timer reset again

        vm.expectRevert(Snow.S__Timer.selector);
        vm.prank(victim);
        snow.earnSnow(); // still blocked
    }
```

Recommended Mitigation: Use a per-user mapping and remove the timer reset from buySnow.

```
-   uint256 private s_earnTimer;
+   mapping(address => uint256) private s_earnTimer;

    function buySnow(uint256 amount) external payable canFarmSnow {
        ...
-       s_earnTimer = block.timestamp;
    }

    function earnSnow() external canFarmSnow {
-       if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks))
↪   {
+       if (s_earnTimer[msg.sender] != 0 && block.timestamp <
↪   (s_earnTimer[msg.sender] + 1 weeks)) {
            revert S__Timer();
        }
        _mint(msg.sender, 1);
-       s_earnTimer = block.timestamp;
+       s_earnTimer[msg.sender] = block.timestamp;
    }
```

Informational

[I-1] `Snow::s_buyFee` should be declared `immutable`

Description: `s_buyFee` is set once in the constructor and never modified. Declaring it `immutable` saves gas on every read and communicates intent clearly.

Recommended Mitigation:

```
-   uint256 public s_buyFee;
+   uint256 public immutable i_buyFee;
```

[I-2] Missing getter functions for key state variables in Snow

Description: `i_farmingOver`, `s_earnTimer`, and `i_weth` have no public getter functions. Users and frontends cannot query the farming end time, per-user earn cooldown, or the WETH token address without off-chain workarounds.

Recommended Mitigation: Add view functions for each variable.

```
function getFarmingOver() external view returns (uint256) { return
↪  i_farmingOver; }
function getEarnTimer(address user) external view returns (uint256) {
↪  return s_earnTimer[user]; }
function getWeth() external view returns (address) { return
↪  address(i_weth); }
```

[I-3] `SnowmanAirdrop::s_claimers` array is declared but never used

Description: `address[] private s_claimers` is declared as a state variable but is never populated with `.push()` and never read anywhere in the contract.

Recommended Mitigation: Either remove the array entirely, or add `s_claimers.push(receiver)` inside `claimSnowman()` if tracking claimers is intended. Also add a getter function.

[I-4] `Snow::collectFee` uses bare `transfer` instead of `safeTransfer`

Description: `i_weth.transfer(s_collector, collection)` is used in `collectFee`, while the rest of the contract uses `SafeERC20`. For non-standard ERC20 tokens that return `false` on failure, this call would silently fail.

Recommended Mitigation:

```
-   i_weth.transfer(s_collector, collection);
+   i_weth.safeTransfer(s_collector, collection);
```

[I-5] `Snowman::SM__NotAllowed` error is declared but never used

Description: `error SM__NotAllowed()` is defined in `Snowman.sol` but is never thrown anywhere in the contract. It is dead code.

Recommended Mitigation: Remove the unused error declaration.

```
-   error SM__NotAllowed();
```